

Chapter 17

Undecidability

We shall prove that not all problems/languages are decidable. In other words there exists languages that are not accepted by any halting Turing machine. The proof uses the diagonalization method to construct a language and then argue that there does not exist any TM that accepts this language.

17.1 A Non Turing Recognizable Language

We use the diagonalization argument to show the existence of a non Turing recognizable language. First let us lay down a few notations and conventions that will be required for our proof.

- Every TM has a finite description (states, transition function, etc.).
- Description of a TM can be thought of as a finite string over $\{0, 1\}$. This can be achieved in many ways. If the description contains k different symbols, then for each symbol associate a string over $\{0, 1\}$ of length $\lceil \log k \rceil$. Now replace every symbol by its corresponding bit string. Hence we get a string over $\{0, 1\}$ for every TM.
- Using the bit string representation described earlier, we can associate a natural number to every bit string in the following natural way. Append a 1 to the left of the string (to distinguish between preceding 0's) and convert it to decimal.
- If a natural number does not correspond to any TM then associate it with some fixed TM (say a TM that accepts all strings).
- By the above scheme, every natural number corresponds to some TM and for every TM there is at least one natural number representing it. For $i \geq 1$, let M_i denote the i -th TM. This gives us a surjective function from natural numbers to the set of *all* TMs.

Theorem 24. *There exists a language that is not Turing recognizable.*

Proof. Let $s_1, s_2, \dots$ be the lexicographic ordering of the strings. Define

$$L_d = \{s_i \mid M_i \text{ does not accept } s_i\}.$$

Suppose L_d is Turing recognizable. Then there is a TM M_j such that $L_d = L(M_j)$ (that is, M_j is the j -th TM in the sequence described above). Now,

$$M_j \text{ accepts } s_j \iff s_j \in L(M_j) \iff s_j \in L_d \iff M_j \text{ does not accept } s_j.$$

Therefore we have a contradiction and L_d is not Turing recognizable. □

17.2 Undecidable Languages

Consider the language

$$A_{TM} = \{\langle M, w \rangle \mid M \text{ accepts } w\}.$$

It is easy to show that A_{TM} is Turing recognizable. Given an input $\langle M, w \rangle$, simulate M on w . If M accepts w then *accept* and if M rejects w then *reject*.

Theorem 25. A_{TM} is undecidable.

Proof. Suppose A_{TM} is decidable. Then there exists a halting TM H that on input $\langle M, w \rangle$, *accepts* if M accepts w and *rejects* if M does not accept w .

Using H , construct another machine N as follows:

Description of Turing machine N

Input: $\langle M \rangle$ where M is a TM

1. Simulate H on $\langle M, \langle M \rangle \rangle$. (A description of H is hardcoded into the machine N .)
2. If H rejects then *accept* else *reject*.

Note that the machine H is a halting TM, hence N is also a halting TM. Now consider what happens when the machine N is provided with the input $\langle N \rangle$.

$$N \text{ accepts } \langle N \rangle \iff H \text{ rejects } \langle N, \langle N \rangle \rangle \iff N \text{ does not accept } \langle N \rangle$$

This is a contradiction. Hence A_{TM} is undecidable. □

17.2.1 Halting Problem

We can show that a language is undecidable by “reducing” a known undecidable language to the given language. Later we will formalize the definition of reduction. But for now let us show the undecidability of the *halting problem* by proving that if the halting problem is decidable then A_{TM} is also decidable. The halting problem is defined as

$$H_{TM} = \{\langle M, w \rangle \mid M \text{ halts on } w\}.$$

Suppose H_{TM} is decidable via a halting TM H . Using H we will construct a halting TM H' for A_{TM} as follows:

Description of Turing machine H'

Input: $\langle M, w \rangle$ where M is a TM and w is a string

1. Simulate H on $\langle M, w \rangle$.
2. If H rejects $\langle M, w \rangle$ then *reject*.
3. If H accepts then simulate M on w .
 - If M accepts then *accept* and if M rejects w then *reject*.

Observe that when we simulate M on w in the above algorithm it will always halt. This is because if M does not halt on w then H rejects in Step 2 itself and we do not proceed to Step 3.

Exercise 32. Show that if A_{TM} is decidable then H_{TM} is decidable (essentially converse of the above construction).

Exercise 33. Show that H_{TM} is Turing recognizable.

Note 11. $\overline{A_{TM}}$ is not Turing recognizable. This is because we know that A_{TM} is Turing recognizable and if $\overline{A_{TM}}$ is also Turing recognizable then it implies that A_{TM} is decidable. The same is true for H_{TM} as well.

This can be generalized to give the following Proposition.

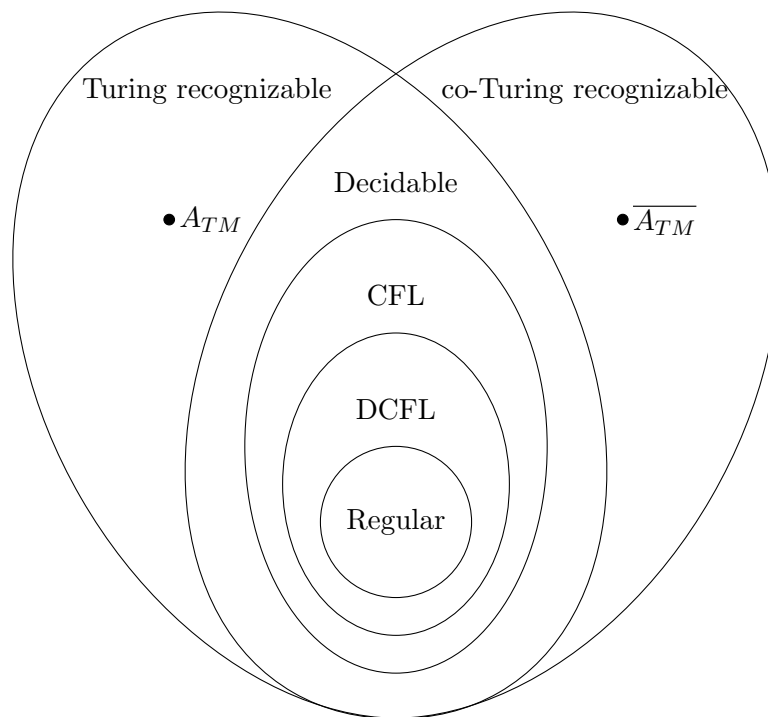
Proposition 26. *If a language L is undecidable and Turing recognizable then $\overline{L}$ is not Turing recognizable.*

17.3 Chomsky Hierarchy

Define

$$\text{co-Turing recognizable} = \{L \mid \overline{L} \text{ is Turing recognizable}\}.$$

The relation between the various classes that we have seen so far can be summarized as follows. This is also known as the Chomsky Hierarchy.



17.4 Reducibility

- Converting one problem to another.
- Examples:

- (a) going around in a city reduces to ability to read a map
- (b) getting good grades reduces to solving problems
- (c) finding max/min reduces to sorting
- (d) multiplication reduces to addition (doing it several times)

17.4.1 Computable Functions

Henceforth, we will also discuss TMs that compute a general function as opposed to outputting just *accept* or *reject*. We will assume that a TM has three tapes, namely

- a read-only *input tape*,
- a read-write *work tape*, and
- a write-only *output tape*.

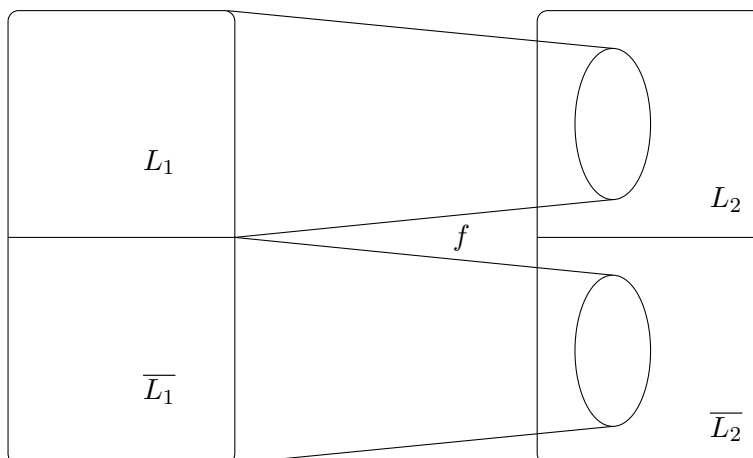
Since the output tape is write-only, the tape head of the output tape is restricted to move only from left to right. There is a special *halting state* in the TM. On any input, as soon as the TM enters the halting state, the computation of the TM stops and whatever string is written on the output tape, is the output of the TM on that particular input. For a TM M and on an input x , we will denote $M(x)$ to be the output of M on x .

Definition 17.4.1. A function $f : \Sigma^* \rightarrow \Sigma^*$ is said to be *computable* if there exists a TM M with output tape, such that for all $x \in \Sigma^*$, $f(x) = M(x)$.

Definition 17.4.2. We say that a language L_1 reduces to a language L_2 if there exists a computable function f , such that for all $x \in \Sigma^*$,

$$x \in L_1 \iff f(x) \in L_2.$$

This is denoted as $L_1 \leq_m L_2$.



- Note that f is a function, hence *every* string gets mapped to some string. But every string in L_2 or $\overline{L_2}$ need not have a pre-image.
- The m in the notation of reduction stands for “many-to-one”.

- If $L_1 \leq_m L_2$ then intuitively it means that L_2 is at least as hard as L_1 . In other words, a solution to L_2 would give a solution to L_1 .
- A halting TM is also known as a *decider*.

Theorem 27. *If $L_1 \leq_m L_2$ and L_2 is decidable then L_1 is decidable.*

Proof. Let f be the computable function such that for all $x \in \Sigma^*$, $x \in L_1 \iff f(x) \in L_2$. Let M be a decider for L_2 . We can then construct a decider for L_1 as follows.

Given $x \in \Sigma^*$, first compute $f(x)$. Simulate M on $f(x)$. If M accepts then *accept* else *reject*. $\square$

Corollary 28. *Let L_1 and L_2 be two languages such that $L_1 \leq_m L_2$.*

- (a) *If L_1 is undecidable then L_2 is undecidable.*
- (b) *If L_1 is not Turing recognizable then L_2 is not Turing recognizable.*

17.5 Examples

1. Consider the language

$$E_{TM} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) = \emptyset\}.$$

Claim 29. $\overline{A_{TM}} \leq_m E_{TM}$.

We will construct a computable function f that takes as input $\langle M, w \rangle$ and produces an output $\langle M' \rangle$ such that M does not accept $w \iff L(M') = \emptyset$.

The reduction function f

Input: $\langle M, w \rangle$

- (a) Construct a TM M' that on input x does the following (note that M' has description of M and w hardcoded into its description):
 - i. If $x \neq w$ then *reject*
 - ii. Else simulate M on w . If M accepts then *accept* and if M rejects then *reject*.

Output: $\langle M' \rangle$

Proof of correctness

Now,

$$\begin{aligned} M \text{ does not accept } w &\implies M' \text{ does not accept any input} \implies L(M') = \emptyset \\ M \text{ accepts } w &\implies M' \text{ accepts only the string } w \implies L(M') = \{w\} \end{aligned}$$

Therefore, M does not accept $w \iff L(M') = \emptyset$ and hence $\overline{A_{TM}} \leq_m E_{TM}$. This proves that E_{TM} is not Turing recognizable.

2. Consider the language

$$EQ_{TM} = \{\langle M_1, M_2 \rangle \mid M_1, M_2 \text{ are TMs and } L(M_1) = L(M_2)\}.$$

Claim 30. $E_{TM} \leq_m EQ_{TM}$.

We will construct a computable function f that takes as input $\langle M \rangle$ and produces an output $\langle M_1, M_2 \rangle$ such that $L(M) = \emptyset \iff L(M_1) = L(M_2)$.

The reduction function f

Input: $\langle M \rangle$

- (a) Set $M_1 := M$.
- (b) Construct a TM M_2 that rejects all inputs (that is $L(M_2) = \emptyset$).

Output: $\langle M_1, M_2 \rangle$

Proof of correctness

Now,

$$L(M) = \emptyset \iff L(M_1) = \emptyset \iff L(M_1) = L(M_2)$$

Therefore, $L(M) = \emptyset \iff L(M_1) = L(M_2)$ and hence $E_{TM} \leq_m EQ_{TM}$. This proves that EQ_{TM} is not Turing recognizable.

3. Consider the language

$$REG_{TM} = \{\langle M \rangle \mid M \text{ is a TM and } L(M) \text{ is regular}\}.$$

Claim 31. $\overline{A_{TM}} \leq_m REG_{TM}$.

We will construct a computable function f that takes as input $\langle M, w \rangle$ and produces an output $\langle M' \rangle$ such that M does not accept $w \iff L(M')$ is regular.

The reduction function f

Input: $\langle M, w \rangle$

- (a) Construct a TM M' that on input x does the following (note that M' has description of M and w hardcoded into its description):
 - i. Simulate M on w .
 - ii. If M accepts w then check if x is of the form $0^n 1^n$ or not. If so then *accept* else *reject*.
 - iii. If M rejects w then *reject*.

Output: $\langle M' \rangle$

Proof of correctness

Now,

$$\begin{aligned}
M \text{ does not accept } w &\implies M' \text{ does not accept any input} \implies L(M') = \emptyset \implies L(M') \text{ is regular} \\
M \text{ accepts } w &\implies L(M') = \{0^n 1^n \mid n \geq 0\} \implies L(M') \text{ is not regular}
\end{aligned}$$

Therefore, M does not accept $w \iff L(M')$ is regular. Hence $\overline{A_{TM}} \leq_m REG_{TM}$. This proves that REG_{TM} is not Turing recognizable.

Similar approach can be used to show that checking whether the language of a TM is a CFL or not is undecidable.

4. Consider the language

$$CON_{TM} = \{\langle M_1, M_2, M_3 \rangle \mid M_1, M_2, M_3 \text{ are TMs and } L(M_1) = L(M_2) \cdot L(M_3)\}.$$

Claim 32. $EQ_{TM} \leq_m CON_{TM}$.

We will construct a computable function f that takes as input $\langle N_1, N_2 \rangle$ and produces an output $\langle M_1, M_2, M_3 \rangle$ such that $L(N_1) = L(N_2) \iff L(M_1) = L(M_2) \cdot L(M_3)$.

The reduction function f

Input: $\langle N_1, N_2 \rangle$

- (a) Set $M_1 := N_1$ and $M_2 := N_2$.
- (b) Construct a TM M_3 that accepts the string ϵ only and rejects every other string.

Output: $\langle M_1, M_2, M_3 \rangle$

Proof of correctness

Now,

$$L(N_1) = L(N_2) \iff L(N_1) = L(N_2) \cdot \{\epsilon\} \iff L(M_1) = L(M_2) \cdot L(M_3)$$

Therefore, $L(N_1) = L(N_2) \iff L(M_1) = L(M_2) \cdot L(M_3)$ and hence $EQ_{TM} \leq_m CON_{TM}$. This proves that CON_{TM} is not Turing recognizable.

Exercise 34. If $L_1 \leq_m L_2$ then $\overline{L_1} \leq_m \overline{L_2}$.

17.6 Rice's Theorem

- A *property* of a language is a function,

$$P : \{\text{set of all languages}\} \longrightarrow \{0, 1\}.$$

- For a language L , if $P(L) = 1$ then we say that L *satisfies* P and if $P(L) = 0$ then we say that L *does not satisfy* P .
- Examples of properties of languages:

- (i) the language contains the string 01001
- (ii) the language is empty
- (iii) the language has exactly 340 strings
- A property P is said to be a *non-trivial* property of languages of Turing machines if there exists two TMs M_1 and M_2 such that $P(L(M_1)) = 0$ and $P(L(M_2)) = 1$.

Theorem 33 (Rice's Theorem). *Let P be a non-trivial property of languages of TMs. Then the language*

$$L_P = \{\langle M \rangle \mid L(M) \text{ satisfies } P\}$$

is undecidable.

Proof. We will show that $A_{TM} \leq_m L_P$.

Case 1 $P(\emptyset) = 0$

Since P is a non-trivial property, there exists a TM N such that $P(L(N)) = 1$. We will construct a computable function f that takes as input $\langle M, w \rangle$ and produces an output $\langle M' \rangle$ such that M accepts $w \iff L(M')$ satisfies P .

The reduction function f

Input: $\langle M, w \rangle$

1. Construct a TM M' that on input x does the following (note that M' has description of M and w hardcoded into its description):
 - (a) Simulate M on w .
 - (b) If M accepts w then simulate N on x . If N accepts then *accept* and if N rejects then *reject*.
 - (c) If M rejects w then *reject*.

Output: $\langle M' \rangle$

Proof of correctness

Now,

$$\begin{aligned} M \text{ accepts } w &\implies L(M') = L(N) \implies L(M') \text{ satisfies } P \\ M \text{ does not accept } w &\implies L(M') = \emptyset \implies L(M') \text{ does not satisfy } P \end{aligned}$$

Therefore $A_{TM} \leq L_P$.

Case 2 $P(\emptyset) = 1$

We can reduce this to Case 1. The complement property $\overline{P}$ is also a non-trivial property of languages of TMs. Since $P(\emptyset) = 1$ therefore $\overline{P}(\emptyset) = 0$. By Case 1 we have $A_{TM} \leq_m L_{\overline{P}}$. Hence $\overline{A_{TM}} \leq_m \overline{L_{\overline{P}}}$ (see Exercise 34). But $\overline{L_{\overline{P}}} = L_P$. Therefore $\overline{A_{TM}} \leq L_P$.

Therefore L_P is undecidable. □

Note 12. Usually in the exam you will be asked to prove that a language is undecidable without using Rice's Theorem. So you must know how to proceed with the argument of such proofs.

Chapter 18

Introduction to Complexity Theory

We next shift our focus to the class of decidable languages and study them in more detail. Specifically, we consider restrictions on computing resources such as time and space, and study the resulting class of problems and their relations.

- From now on we will consider only halting Turing machines unless otherwise specified.
- We will assume that our TMs have 3 tapes - a read only input tape, a read/write work tape and a write only output tape.

18.1 Time and Space Complexity

Definition 18.1.1. Let M be a Turing machine.

- The *running time* of M (also known as *time complexity*) is a function $f : \mathbb{N} \rightarrow \mathbb{N}$, such that for all inputs x with $|x| = n$, M halts on x within $f(n)$ number of steps.
- The *space required* by M (also known as *space complexity*) is a function $f : \mathbb{N} \rightarrow \mathbb{N}$, such that for all inputs x with $|x| = n$, M 's work tape uses at most $f(n)$ cells on input x .

Note 13. A few key points when dealing with time/space complexity.

- Generally we choose the best (smallest) possible function $f(n)$ when describing the time or space complexity of a TM.
- When considering time or space complexity of a TM we use the asymptotic notations and disregard multiplicative constants and lower order terms.

Lets take a small detour and refresh ourselves with the asymptotic notations for comparing functions.

18.1.1 Asymptotic Analysis of Functions

Let $f, g : \mathbb{N} \rightarrow \mathbb{N}$.

- $f(n) \in O(g(n))$ if $\exists c > 0$ and $n_0 \geq 0$, such that for all $n \geq n_0$, $f(n) \leq c \cdot g(n)$.
- $f(n) \in \Theta(g(n))$ if $\exists c_1, c_2 > 0$ and $n_0 \geq 0$, such that for all $n \geq n_0$,
 $c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$.
- $f(n) \in o(g(n))$ if $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.

18.1.2 Deterministic Time and Space Classes

Definition 18.1.2 (Deterministic Complexity Classes).

- Let $t : \mathbb{N} \rightarrow \mathbb{N}$.

$$\text{TIME}(t(n)) = \{L \mid L = L(M) \text{ for some TM } M \text{ having } O(t(n)) \text{ time complexity}\}.$$

- Let $s : \mathbb{N} \rightarrow \mathbb{N}$.

$$\text{SPACE}(s(n)) = \{L \mid L = L(M) \text{ for some TM } M \text{ having } O(s(n)) \text{ space complexity}\}.$$

18.2 Time and Space Complexity of Nondeterministic TMs

Definition 18.2.1. Let N be a nondeterministic Turing machine.

- The *running time* of N (also known as *time complexity*) is a function $f : \mathbb{N} \rightarrow \mathbb{N}$, such that for all inputs x , every computation path of N halts within $f(|x|)$ number of steps.
- The *space required* by M (also known as *space complexity*) is a function $f : \mathbb{N} \rightarrow \mathbb{N}$, such that for all inputs x , every computation path of N uses at most $f(|x|)$ cells on its work tape.

Theorem 34. Let $t(n) \geq n$. Then every $t(n)$ time TM having multiple work tapes has an equivalent $O((t(n))^2)$ time TM with a single work tape.

Theorem 35. Let $t(n) \geq n$. Then every $t(n)$ time NDTM has an equivalent $2^{O(t(n))}$ time deterministic TM.

18.2.1 Nondeterministic Time and Space Classes

Definition 18.2.2 (Nondeterministic Complexity Classes).

- Let $t : \mathbb{N} \rightarrow \mathbb{N}$.

$$\text{NTIME}(t(n)) = \{L \mid L = L(N) \text{ for some NDTM } N \text{ having } O(t(n)) \text{ time complexity}\}.$$

- Let $s : \mathbb{N} \rightarrow \mathbb{N}$.

$$\text{NSPACE}(s(n)) = \{L \mid L = L(N) \text{ for some NDTM } N \text{ having } O(s(n)) \text{ space complexity}\}.$$

18.3 The Complexity Classes P and NP

In this section we will define the complexity classes P and NP and see examples of problems in these classes.

18.3.1 The Class P

- The class of problems that can be solved in polynomial time by a deterministic TM.
- Formally,

$$P = \bigcup_{k \geq 0} \text{TIME}(n^k).$$

- Widely accepted as the class of problems with an efficient solution. Most algorithms that we have seen so far belong to P.
- Examples: Matrix multiplication, graph reachability, all CFLs (using dynamic programming), computing determinant of a matrix using the Gaussian elimination method.

Note 14. Time complexity of a TM (or algorithm) is a property of the design of the TM and not that of the problem. For the same problem we can have TMs of differing time complexity.

18.3.2 The Class NP

- The class of problems that can be solved in polynomial time by a nondeterministic TM.
- Formally,

$$NP = \bigcup_{k \geq 0} \text{NTIME}(n^k).$$

- Widely accepted as the class of problems whose solutions are efficiently verifiable. Based on this we have the following alternative definition of NP.

Definition 18.3.1 (Alternate Definition of NP). $L \in NP$ if there exists constants $c > 0$ and $k \geq 0$, and a polynomial time TM V (also known as the “verifier”), such that $\forall x \in \Sigma^*$,

$$x \in L \iff \exists y \in \Sigma^* \text{ such that } |y| \leq c \cdot |x|^k \text{ and } V(x, y) = 1.$$

- The string y is also known as the *certificate* or *proof* of the fact that x belongs to L .
- Its length is polynomially bounded by the length of x .
- If $x \in L$ then there exists *some* certificate which makes a verifier accept. Alternately, if $x \notin L$ then *every* certificate must make the verifier reject.
- The certificate can be any string as long as it helps the verifier make the correct decision.
- Generally we will use this definition to show that a language is in NP.

18.3.3 Examples of Problems in NP

1.

$$\text{COMPOSITES} = \{p \mid p \text{ is a composite number}\}.$$

Claim 36. COMPOSITES $\in$ NP.

Certificate

A positive integer q .

Verifier's Algorithm

Input: $\langle p, q \rangle$

- (a) Check whether $1 < q < p$. If not, then *reject*.
- (b) Check if q divides p . If yes, then *accept* else *reject*.

Proof of correctness

p is a composite number if and only if there exists a number q strictly between 1 and p such that q divides p .

2. Let G_1 and G_2 be two graphs. For a graph G let $V(G)$ and $E(G)$ denote the vertex set and edge set of G respectively. We say G_1 is *isomorphic* to G_2 (denoted as $G_1 \cong G_2$) if there is a bijective function

$$f : V(G_1) \longrightarrow V(G_2),$$

such that for all pair of vertices $u, v \in V(G_1)$,

$$(u, v) \in E(G_1) \iff (f(u), f(v)) \in E(G_2).$$

Then,

$$\text{GI} = \{\langle G_1, G_2 \rangle \mid G_1 \cong G_2\}.$$

Claim 37. $\text{GI} \in \text{NP}$.

Certificate

A function $f : V(G_1) \longrightarrow V(G_2)$.

Verifier's Algorithm

Input: $\langle G_1, G_2, f \rangle$

- (a) Check that f is indeed a bijective function from $V(G_1)$ to $V(G_2)$. If not, then *reject*.
- (b) $\forall u, v \in V(G_1)$ check whether the following statement is true

$$(u, v) \in E(G_1) \iff (f(u), f(v)) \in E(G_2).$$

- (c) If the check succeeds for all pairs then *accept* and if it fails for even one pair then *reject*.

3.

$$\text{Clique} = \{\langle G, k \rangle \mid G \text{ has a complete subgraph of size at least } k\}.$$

Claim 38. $\text{Clique} \in \text{NP}$.

Certificate

$U \subseteq V(G)$.

Verifier's Algorithm

Input: $\langle G, k, U \rangle$

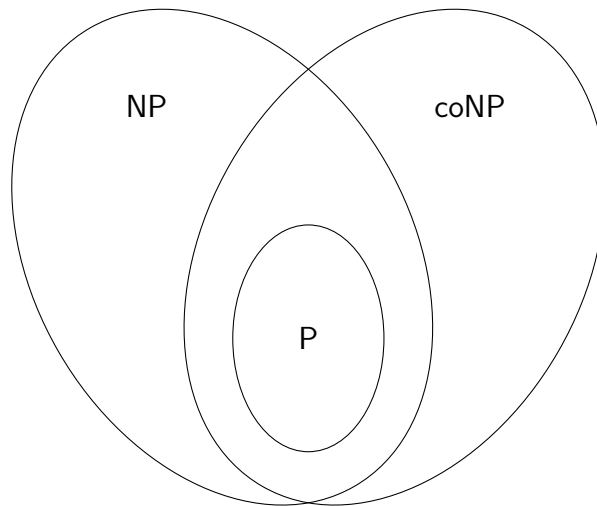
- (a) Verify that U is indeed a subset of $V(G)$. If not, then *reject*.
- (b) Verify that $|U| = k$. If not, then *reject*.
- (c) $\forall u, v \in U$ check whether $(u, v) \in E(G)$. If the check succeeds for all pairs then *accept* else *reject*.

18.3.4 The Class coNP

- It is the class of problems whose complement is in NP.
- Formally,

$$\text{coNP} = \{L \mid \overline{L} \in \text{NP}\}.$$

- The following diagram illustrates the currently known relation between P, NP and coNP.



- It is an open question whether $\text{NP} = \text{coNP}$ or not.
- Note that P is a deterministic class and hence is closed under complementation.

Exercise 35. Show that if $\text{NP} \subseteq \text{coNP}$ then $\text{NP} = \text{coNP}$.

Chapter 19

NP Completeness

19.1 Definition of NP-Completeness

We again shift our focus back to reductions. Specifically we consider a type of reductions known as polynomial time (or polytime) reductions.

Definition 19.1.1. A language L_1 *polynomial time reduces* to a language L_2 (denoted as $L_1 \leq_p L_2$) if there is a polynomial time computable function $f : \Sigma^* \rightarrow \Sigma^*$ such that for all $x \in \Sigma^*$,

$$x \in L_1 \iff f(x) \in L_2.$$

Definition 19.1.2. - A language L is said to be **NP-hard** if for all languages $L' \in \text{NP}$ we have that $L' \leq_p L$.

- A language L is said to be **NP-complete** if L is NP-hard and $L \in \text{NP}$.

Therefore, in some sense NP-complete problems are the hardest problems present in NP.

Note 15. A reduction should always be of “lesser power” than the complexity class under consideration. Otherwise one can always cheat by solving the problem first by using the power of reduction and then mapping it to a trivial Yes/No instance.

19.2 Properties of NP-Completeness

Theorem 39. If L is NP-complete and $L \in \text{P}$ then $\text{P} = \text{NP}$.

Proof. Let L' be a language in NP. Since L is NP-complete therefore $L' \leq_p L$ via a polynomial time computable function, say f . Also let M be a polynomial time TM for L (since $L \in \text{P}$).

We construct a polynomial time machine M' for L' . On input x , M' first constructs $f(x)$ then simulates M on $f(x)$ and outputs whatever M outputs.

Since f is polynomial time computable and M is a polynomial time TM therefore M' is also a polynomial time TM. Now,

$$x \in L' \iff f(x) \in L \iff M \text{ accepts } f(x) \iff M' \text{ accepts } x.$$

Therefore $\text{NP} \subseteq \text{P}$. □

Theorem 40. If A is NP-hard and $A \leq_p B$ then B is NP-hard.

We first prove the following Lemma.

Lemma 41. *If $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$ then $L_1 \leq_p L_3$.*

Proof. Suppose $L_1 \leq_p L_2$ and $L_2 \leq_p L_3$ via polynomial time computable functions, say f and g respectively.

We construct a polynomial time computable function h that on input x outputs $h(x) = g(f(x))$. On input x , first compute $f(x)$ using the polytime machine for f . Then compute $g(f(x))$ using the polytime machine for g . Output $g(f(x))$.

Since f and g are polynomial time computable therefore h is also a polynomial time computable (sum of two polynomials is a polynomial). Now,

$$x \in L_1 \iff f(x) \in L_2 \iff g(f(x)) \in L_3 \iff h(x) \in L_3.$$

Therefore $L_1 \leq_p L_3$. □

Proof of Theorem 40. Let L be a language in NP. Since A is NP-hard therefore $L \leq_p A$. Also we have that $A \leq_p B$. Applying Lemma 41 we have $L \leq_p B$. Therefore B is also NP-hard. □

19.3 SAT is NP-complete

19.3.1 Terminology and Notations

- *Boolean variables* are variables that take the value 0 or 1.
- *Boolean operators*: AND ($\wedge$), OR ($\vee$) and NOT ($\neg$)
- A *Boolean formula* is an expression that is defined inductively over Boolean variables and Boolean operators.
- Examples
 - (i) $\phi_1 : (x \wedge \neg y) \vee \neg x$
 - (ii) $\phi_2 : (x \vee y) \wedge (\neg x \vee z) \wedge (\neg y \vee \neg z)$
- A *truth assignment* τ is an assignment of values 0 or 1 to every variable. Specifically, $\tau : \{x_1, x_2, \dots, x_n\} \longrightarrow \{0, 1\}$.
- Evaluation of a Boolean formula ϕ on a truth assignment τ is the assignment of the values of τ to the variables in ϕ so that it evaluates to 0 or 1. It is denoted as $\phi(\tau)$.
- Example of a truth assignment, τ , for ϕ_2 above. $\tau(x) = 1, \tau(y) = 0, \tau(z) = 0$. Then $\phi_2(\tau) = 0$ since the second clause evaluates to 0.
- A Boolean formula ϕ is said to be *satisfiable* if there exists a truth assignment τ such that $\phi(\tau) = 1$.
- $\text{SAT} = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable Boolean formula}\}$.
- A *literal* is a variable x or its negation $\neg x$.
- A *clause* is an OR of literals.

- A *CNF formula* is an AND of clauses.
- A *3CNF formula* is CNF formula where every clause has exactly 3 literals.
- $3SAT = \{\langle \phi \rangle \mid \phi \text{ is a satisfiable 3CNF Boolean formula}\}.$

Exercise 36. Show that SAT is in NP.

19.3.2 Cook-Levin Theorem

We next show that SAT is NP-hard. This was proven independently by Stephen Cook and Leonid Levin in 1971 and 1973 respectively. It is one the fundamental results in computer science and the P versus NP problem originated from this result.

Theorem 42 (Cook-Levin Theorem). *SAT is NP-hard.*

Proof Idea. Let $L \in \text{NP}$ accepted by a NDTM N . Suppose the running time on N is n^k on a input of length n . Note that the machine N can visit at most n^k cells on its tape. Given an input x to N such that $|x| = n$, we will construct a Boolean formula ϕ such that $x \in L$ if and only if $\phi \in \text{SAT}$.

Consider an $n^k \times n^k$ matrix, say M , whose i -th row stores the i -th configuration of N on x . Each element of M is a symbol from the set $C := Q \cup \Gamma$, that is, it contains a state symbol or a tape symbol.

Remark. For a NDTM N , every $n^k \times n^k$ matrix corresponds to *some* computation path of N on an input x .

- Construct a formula ϕ_{cell} which is satisfiable if and only if every cell of the matrix holds exactly one symbol. Define Boolean variables $x_{i,j,s}$, where $1 \leq i, j \leq n^k$ and $s \in C$, such that, $x_{i,j,s} = 1$ if and only if $M(i, j)$ contains s . Then

$$\phi_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left(\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{s, t \in C, s \neq t} (\neg x_{i,j,s} \vee \neg x_{i,j,t}) \right) \right).$$

Essentially the above formula encodes the fact that for cell (i, j) , $M(i, j)$ stores some symbol s and $M(i, j)$ does not store two different symbols s and t .

- Next construct a formula ϕ_{start} which is satisfiable if and only if the first row of M encodes the start configuration. That is,

$$\phi_{\text{start}} = x_{1,1,q_0} \wedge x_{1,2,x_1} \wedge x_{1,3,x_2} \wedge \dots \wedge x_{1,n+1,x_n} \wedge x_{1,n+2,\sqcup} \dots \wedge x_{1,n^k,\sqcup}.$$

- Next construct a formula ϕ_{accept} which is satisfiable if and only if some row of M encodes an accept configuration. That is,

$$\phi_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}.$$

- Finally we define a formula ϕ_{move} which is satisfiable if and only if the $(i+1)$ -th row follows legally (according to the transition function of N) from the i -th row of M . To check this we consider every possible 2×3 window in M and verify whether its a legal window (that is

the bottom row follows legally from the top row). We consider a window of dimension 2×3 because (i) a configuration depends only on the previous configuration (hence we consider two rows in every window), and (ii) between two successive configurations only left cell or the right cell of a given cell gets modified (hence we consider three columns in every window). Hence define

$$\phi_{move} = \bigwedge_{\substack{1 \leq i < n^k \\ 1 < j < n^k}} \left(\bigvee_{\substack{a_1, \dots, a_6 \\ \text{is a legal} \\ \text{window}}} (x_{i,j-1,a_1} \wedge x_{i,j,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j-1,a_4} \wedge x_{i+1,j,a_5} \wedge x_{i+1,j+1,a_6}) \right).$$

- The final formula ϕ is

$$\phi = \phi_{cell} \wedge \phi_{start} \wedge \phi_{accept} \wedge \phi_{move}.$$

The key property that we use in the above construction is that computation of a TM happens *locally*. This allows us to restrict ourselves to a constant sized window at every step.

Exercise 37. Verify that the size of ϕ constructed in the above proof is $O(n^{2k})$. Also verify that the construction takes polynomial time.

□

Exercise 38. Show that $\text{SAT} \leq_p 3\text{SAT}$.

19.4 More NP-complete Problems

1. Clique is NP-complete.

- Let $G = (V, E)$ be a graph. A subset $S \subseteq V$ forms a *clique* if for every pair of vertices $u, v \in S$, $(u, v) \in E$.
- $\text{Clique} = \{\langle G, k \rangle \mid G \text{ has a clique of size at least } k\}$.

Theorem 43. *Clique is NP-complete.*

(a) Showing that the problem is in NP.

Exercise 39. Show that *Clique* is in NP.

(b) Choosing a suitable NP-complete problem. We will show that

$$3\text{SAT} \leq_p \text{Clique}.$$

(c) The reduction. Let ϕ be an instance of 3SAT where

$$\phi := (l_{11} \vee l_{12} \vee l_{13}) \wedge (l_{21} \vee l_{22} \vee l_{23}) \wedge \dots \wedge (l_{m1} \vee l_{m2} \vee l_{m3}).$$

Construct a graph $G_\phi = (V, E)$ as follows:

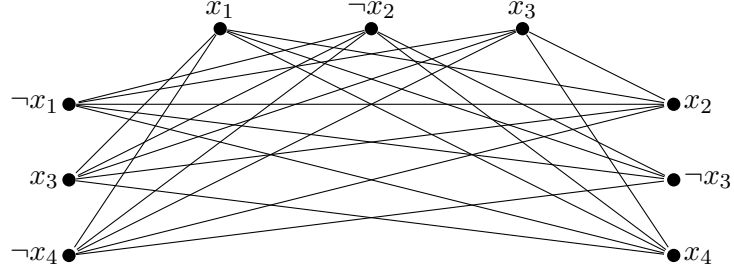
$$\begin{aligned} V &= \{v_{ij} \mid 1 \leq i \leq m, 1 \leq j \leq 3\} && \text{i.e. one vertex per literal} \\ E &= \{(v_{ij}, v_{i'j'}) \mid i \neq i' \text{ and } l_{ij} \neq \neg l_{i'j'}\} \end{aligned}$$

In other words, we place an edge between two vertices if their corresponding literals belong to different clauses and one literal is not the negation of the other literal.

Set $k = m$.

Given below is an example of this construction.

$$\phi := (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee x_3 \vee \neg x_4) \wedge (x_2 \vee \neg x_3 \vee x_4)$$



Graph G_ϕ

(d) The construction of G_ϕ from ϕ can be achieved in quadratic time in the length of ϕ .

(e) Proof of correctness.

If ϕ is satisfiable then there exists a truth assignment τ such that each clause has at least one satisfying literal. Let $L = \{l_1, l_2, \dots, l_m\}$ be a set of m satisfying literals, one from each clause. And $U = \{v_1, v_2, \dots, v_m\}$ be the corresponding vertices in the graph G_ϕ . Since every literal $l_i \in L$ belongs to a different clause and every literal is a satisfying literal, therefore it must be that for every $i \neq j$, $l_i \neq \neg l_j$. Hence for every $i \neq j$, (v_i, v_j) is an edge in E . Therefore the subgraph induced by U is a complete subgraph.

For the other direction, let G_ϕ have a complete subgraph of size at least k . Since there are no edges in G_ϕ between the vertices of the same clause and since the number of clauses is $m (= k)$ therefore G_ϕ cannot have a complete subgraph of size strictly more than k . Let $L = \{l_1, l_2, \dots, l_m\}$ be the literals corresponding to the complete subgraph. We will construct a satisfying assignment as follows. For a literal $l \in L$, if $l = x_i$ then set $x_i = 1$ and if $l = \neg x_i$ then set $x_i = 0$. Other variables can be assigned any value 0 or 1. This assignment is consistent because both a literal l and $\neg l$ cannot be in the set L because L corresponds to a complete graph. Also since every literal in L is from a different clause, the above assignment is a satisfying assignment for ϕ .

2. IndSet is NP-complete.

- Let $G = (V, E)$ be a graph. A subset $S \subseteq V$ is an *independent set* if for every pair of vertices $u, v \in S$, $(u, v) \notin E$.
- $\text{IndSet} = \{\langle G, k \rangle \mid G \text{ has an independent set of size at least } k\}$.

Theorem 44. *IndSet is NP-complete.*

(a) Showing that the problem is in NP.

Exercise 40. Show that IndSet is in NP.

(b) Choosing a suitable NP-complete problem. We will show that

$$\text{Clique} \leq_p \text{IndSet}.$$

- (c) The reduction. Let $\langle G = (V, E), k \rangle$ be an instance of **Clique**. We will construct an instance of **IndSet**, $\langle G' = (V', E'), k' \rangle$ as follows:

$$\begin{aligned} V' &= V \\ E' &= \{(u, v) \mid (u, v) \notin E\} \end{aligned}$$

In other words, G' is the complement of the graph G .

Set $k' = k$.

- (d) The construction of G' can be achieved in linear time from the graph G .

- (e) Proof of correctness.

G has a clique of size m if and only if the same set of vertices forms an independent set of size m in G' .

3. **VertexCover** is NP-complete.

- Let $G = (V, E)$ be a graph. A subset $S \subseteq V$ is a *vertex cover* if every edge in G has at least one of its endpoints in S .
- $\text{VertexCover} = \{\langle G, k \rangle \mid G \text{ has a vertex cover of size at most } k\}$.

Exercise 41. Show that **VertexCover** is in NP-complete.

4. **HamPath** is NP-complete.

- Let $G = (V, E)$ be a directed graph. A *Hamiltonian path* in G is a path that visits every vertex exactly once.
- $\text{HamPath} = \{\langle G, s, t \rangle \mid G \text{ is a directed graph having a Hamiltonian path from } s \text{ to } t\}$.

Theorem 45. **HamPath** is NP-complete.

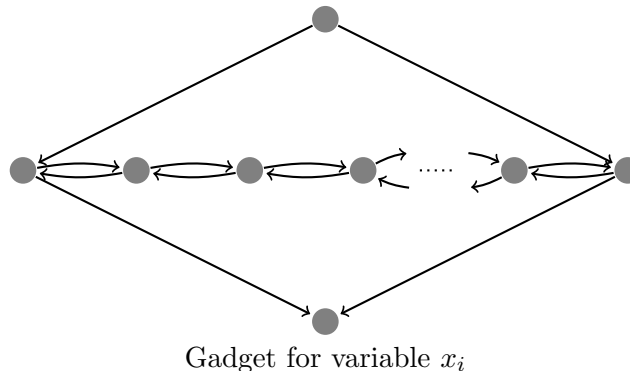
- (a) Showing that the problem is in NP.

Exercise 42. Show that **HamPath** is in NP.

- (b) Choosing a suitable NP-complete problem. We will show that

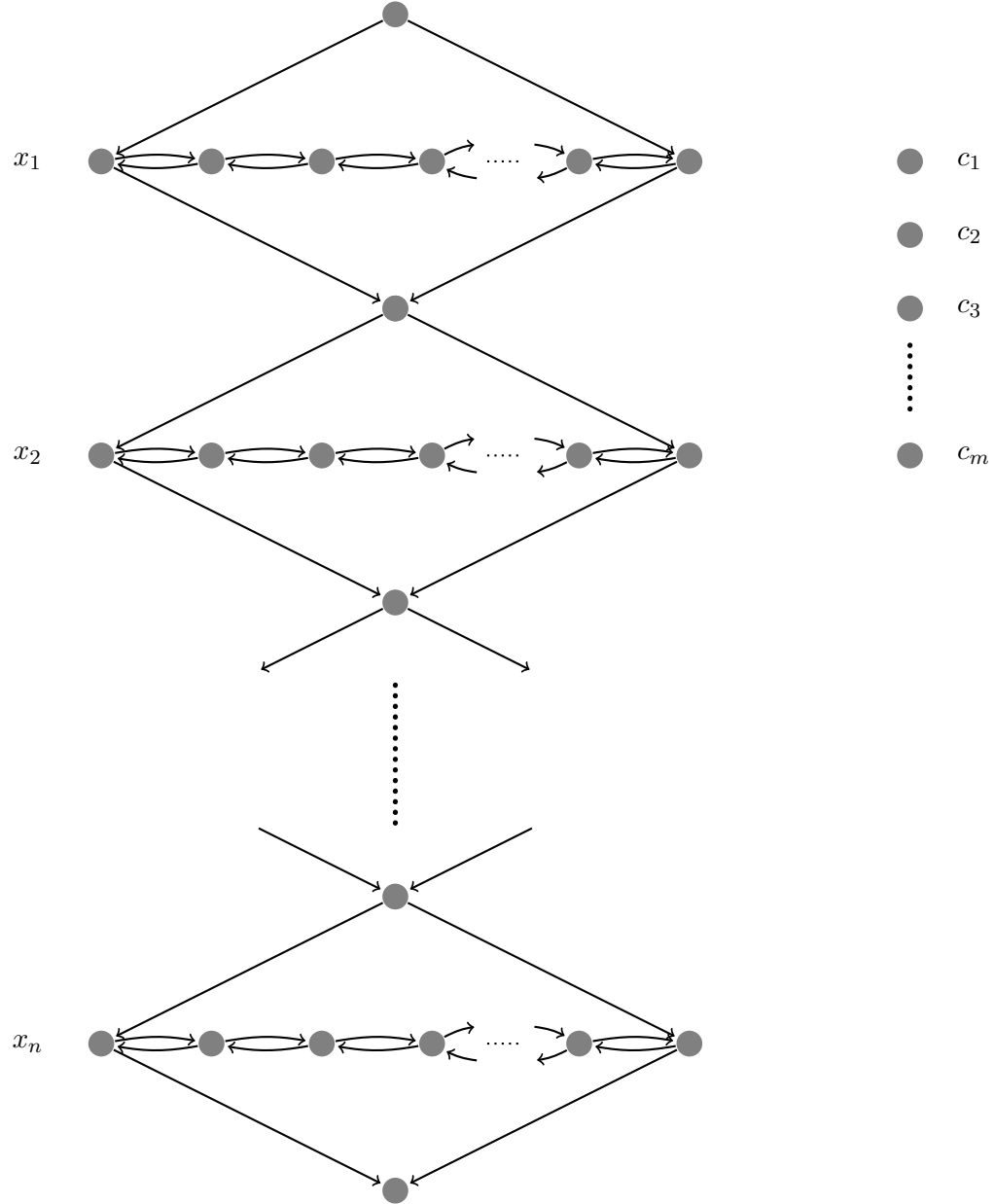
$$3\text{SAT} \leq_p \text{HamPath}.$$

- (c) The reduction. Let ϕ be an instance of **3SAT** having n variables $x_1, \dots, x_n$ and m clauses $c_1, \dots, c_m$. Construct the following gadget corresponding to each variable.

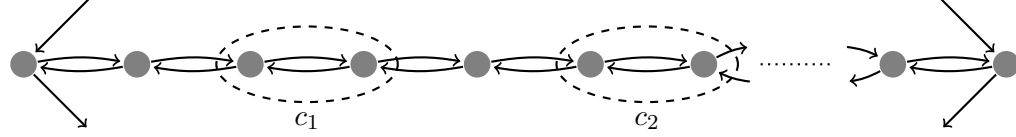


The number of vertices that appear on the horizontal row is $3m + 1$ excluding the two end vertices. For every clause we add a single vertex.

The gadgets corresponding to the variables are connected as shown in the figure below together with the vertices for the m clauses.

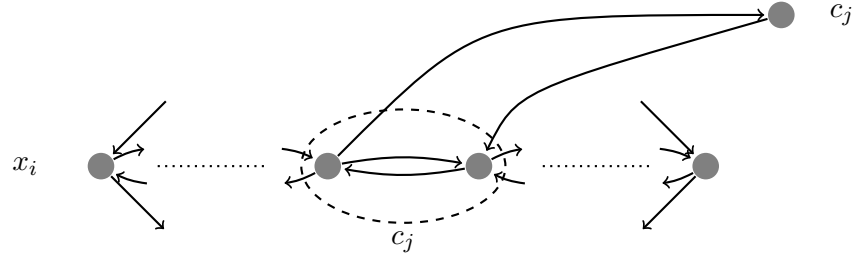


The horizontal row of a variable gadget has $3m + 1$ vertices with two adjacent vertices for each clause and a single *buffer vertex* in the middle. The following figure gives a better explanation.



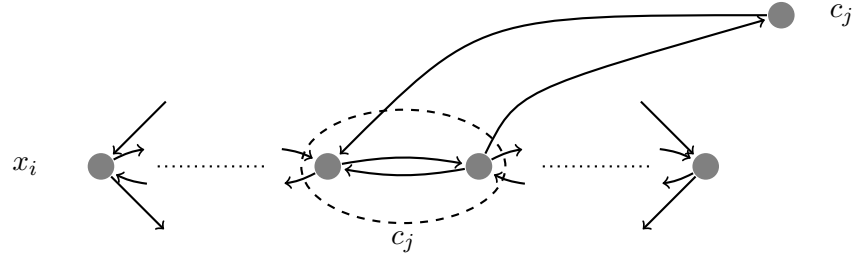
A variable gadget with the vertices for each clause

If a variable x_i appears in a clause c_j in its positive form we connect the clause gadget to its corresponding pair of variables in the variable gadget by the following two edges (shown in the figure below).



When clause c_j contains x_i

Now if a variable x_i appears in a clause c_j in its negated form we connect the clause gadget to its corresponding pair of variables in the variable gadget by the following two edges, say *clause edges* (shown in the figure below).



When clause c_j contains $\neg x_i$

Set s as the top vertex in the gadget corresponding to x_1 and t as the bottom vertex in the gadget corresponding to x_n . This ends the construction of the graph corresponding to the formula ϕ . Let us name the graph G_ϕ .

(d) The construction of G_ϕ from ϕ can be achieved in time $O(mn)$ which is also the size of G_ϕ .

(e) Proof of correctness.

If ϕ is satisfiable, then there is a truth assignment τ such that $\phi(\tau) = 1$. We describe a path from s to t that traverses every vertex in G_ϕ .

- We traverse a variable gadget starting from its top vertex to its bottom vertex in the following manner.
 - If $\tau(x_i) = 1$ we traverse the horizontal row of the gadget corresponding to x_i from left to right.
 - If $\tau(x_i) = 0$ we traverse the horizontal row of the gadget corresponding to x_i from right to left.

This path covers all the vertices corresponding to the n variable gadgets.

- Since τ is a satisfying assignment, every clause in ϕ has at least one true literal with respect to τ . Let l_j be the true literal corresponding to a clause c_j . When traversing the variable gadget corresponding to the vertex of l_j , we do a detour and cover the vertex corresponding to c_j using the two clause edges. Note that if $l_j = x_i$ for some variable x_i then we would be traversing the horizontal row of x_i from left to right and if $l_j = \neg x_i$ for some variable x_i then we would be traversing the horizontal row of x_i from right to left. Hence by taking the detour we do not disturb the property that every vertex is traversed exactly once.

This gives a Hamiltonian path from s to t in G_ϕ .

On the other hand, suppose there is a Hamiltonian path from s to t in the graph G_ϕ .

- We first argue that this path cannot jump from one variable gadget to another variable gadget using a clause vertex. Suppose the path jumps out of a vertex u in a variable gadget to a clause vertex and then jumps in to a vertex in a different variable gadget. Then the buffer vertex after u can never be covered by the path as there is no way to reach that vertex. Hence the Hamiltonian path traverses every variable gadget completely from left to right or from right to left.
- We next define a truth assignment for ϕ . If the variable gadget corresponding to the variable x_i is traversed from left to right then set $x_i = 1$ and if it is traversed from right to left then set $x_i = 0$. It is easy to verify that this is indeed a satisfying assignment.

5. UHamPath is NP-complete.

- Let $G = (V, E)$ be an undirected graph. A *Hamiltonian path* in G is a path that visits every vertex exactly once.
- $\text{UHamPath} = \{\langle G, s, t \rangle \mid G \text{ is undirected and has a Hamiltonian path from } s \text{ to } t\}$.

Exercise 43. Show that UHamPath is in NP-complete.

(Hint: Reduce HamPath to UHamPath. Don't see the proof from your textbook before giving it a try.)

Chapter 20

Space Complexity

Recall the following definitions.

$$\begin{aligned}\text{SPACE}(f(n)) &= \{L \mid L = L(M) \text{ for some det. TM } M \text{ that uses } O(f(n)) \text{ amount of space}\} \\ \text{NSPACE}(f(n)) &= \{L \mid L = L(N) \text{ for some NDTM } N \text{ that uses } O(f(n)) \text{ amount of space}\}\end{aligned}$$

Note:

- The space used by a TM is the number of cells used in the work tape of the TM.
- The space used by a NDTM is the maximum space used over all computation paths of the TM.

20.1 Basic Relations Between the Time and Space Classes

Claim 46. (a) $\text{DTIME}(f(n)) \subseteq \text{SPACE}(f(n))$ and

(b) $\text{NTIME}(f(n)) \subseteq \text{NSPACE}(f(n))$.

If a TM runs for t steps on a input then it cannot visit more than t cells.

Claim 47. $\text{SPACE}(f(n)) \subseteq \text{NSPACE}(f(n)) \subseteq \text{DTIME}(2^{O(f(n))})$

The first containment follows from the definition of nondeterminism.

Every cell of a TM (deterministic or non deterministic) can contain c different symbols, where c is a constant that depends on the description of the TM. Now if the TM uses $kf(n)$ cells on an input of length n then the total number of possible configurations of the TM is $c^{kf(n)}$. Now design a deterministic TM that does a DFS/BFS on the configuration graph and accepts if and only if there is a path from the start configuration to an accept configuration. Since the size of the graph is at most $c^{kf(n)}$ therefore the running time is $c^{O(f(n))}$ which is also $2^{O(f(n))}$.

Note 16. For every TM (deterministic or non deterministic) there is a constant K , such that if the TM uses s cells to compute its answer then it must produce the answer within K^s number of steps. This is because, if the TM does not produce an answer within K^s number of steps then there must be some configuration which is visited twice during the computation and hence the machine gets into an infinite loop.

One can get rid of such a scenario by maintaining a counter in the TM that counts upto K^s , and if the machine has not accepted or rejected its input by that time, then it rejects the input.

Claim 48. $\text{NTIME}(f(n)) \subseteq \text{SPACE}(f(n))$

Consider the configuration graph of a NDTM N on an input x , that takes t steps to produce an answer. The maximum length of any path in $G_{N,x}$ from the root vertex (start config.) to a leaf (halting config.) is at most t . Note that since the running time of N is t therefore $G_{N,x}$ has $2^{O(t)}$ vertices in it. Hence a standard traversal algorithm will not work.

We perform a “non-marking DFS” on $G_{N,x}$ to check for reachability. Starting from the root, we implement DFS using a stack that stores the path from the root to the current vertex in the graph. Whenever all children of a vertex v gets visited, we go back to the parent of v and move to the lexicographically next sibling of v . Whenever we visit a new vertex we push it onto the stack and whenever we are done visiting all children of a vertex, we pop it out of the stack.

Note that the above algorithm does not mark a vertex once it is visited. Hence a vertex can get visited several times and the running time of the algorithm can be as large as $2^{O(t)}$, although the number of vertices can be much smaller than that. The only catch in the above algorithm is if the configuration graph contains cycles. Since we are not marking the vertices we can run into an infinite loop. However the graph can be made acyclic by a little bit of preprocessing, as mentioned in Note 16.

20.1.1 A Simple Example

Claim 49. $\text{SAT} \in \text{SPACE}(n)$.

Input: A Boolean formula ϕ having n variables.

1. Iterate over all truth assignments $\tau \in \{0, 1\}^n$.
2. Compute $\phi(\tau)$, by substituting the values in each variable of ϕ .
3. If $\phi(\tau) = 1$ for some τ then *accept*.
4. If for all τ , $\phi(\tau) = 0$ then *reject*.

The algorithm needs space to store a truth assignment and to compute $\phi(\tau)$. Both can be implemented in linear space in the size of the formula.

20.1.2 Some Standard Classes and their Relations

- $\text{L} = \text{SPACE}(\log n)$
- $\text{NL} = \text{NSPACE}(\log n)$
- $\text{PSPACE} = \bigcup_{k \geq 0} \text{SPACE}(n^k)$
- $\text{NPSPACE} = \bigcup_{k \geq 0} \text{NSPACE}(n^k)$
- $\text{EXP} = \bigcup_{k \geq 0} \text{TIME}(2^{n^k})$
- $\text{NEXP} = \bigcup_{k \geq 0} \text{NTIME}(2^{n^k})$

Known Relations

$$\text{L} \xrightarrow{\text{defn}} \text{NL} \xrightarrow{\text{Claim 47}} \text{P} \xrightarrow{\text{defn}} \text{NP} \xrightarrow{\text{Claim 48}} \text{PSPACE} \xrightarrow{\text{defn}} \text{NPSPACE} \xrightarrow{\text{Claim 47}} \text{EXP} \xrightarrow{\text{defn}} \text{NEXP}$$

20.2 Savitch's Theorem

Theorem 50 (Savitch's Theorem). *For $f(n) \geq \log n$, $\text{NSPACE}(f(n)) \subseteq \text{SPACE}(f(n)^2)$.*

Proof. Let $L \in \text{NSPACE}(f(n))$ accepted by a NDTM say N . Given an input x , consider the configuration graph of N on x , $G_{N,x}$. The number of vertices in the graph is $2^{O(f(n))}$. To show that L is in $\text{SPACE}(f(n)^2)$, it is enough to give a deterministic algorithm that solves the reachability problem in a graph with n vertices, using $O(\log^2 n)$ space.

We give a recursive algorithm to solve reachability, that is based on the following idea.

There is a path from s to t in G of length at most n if and only if there exists a vertex $u \in G$ such that there is a path from s to u in G of length at most $n/2$ and there is a path from v to t in G of length at most $n/2$.

Note that in a graph with n vertices the maximum length of a path between any two vertices (if it exists) is n . We define a recursive algorithm **RecReach** as follows:

$$\begin{aligned} \text{RecReach}(G, s, t, i) &= 1 && \text{if there is a path from } s \text{ to } t \text{ in } G \text{ of length at most } 2^i \\ &= 0 && \text{otherwise} \end{aligned}$$

Description of $\text{RecReach}(G, s, t, i)$

- (i) If $i = 0$ then output 1 if and only if $s = t$ or if there is an edge from s to t .
- (ii) Output 1 if there exists a vertex $u \in G$ such that
 - $\text{RecReach}(G, s, u, i - 1) = 1$ and $\text{RecReach}(G, u, t, i - 1) = 1$.
- (iii) If there is no vertex that satisfies the above condition then output 0.

Let $S(n, i)$ be the space used by $\text{RecReach}(G, s, t, i)$ where G has n vertices and the distance between s and t is at most 2^i . Then,

$$\begin{aligned} S(n, \log n) &= S(n, \log n - 1) + c_1 \log n \\ S(n, 0) &= c_2. \end{aligned}$$

Solving the recurrence we have $S(n, \log n) = O(\log^2 n)$. □

Corollary 51. 1. $\text{NL} \subseteq \text{SPACE}(\log^2 n)$.

2. $\text{PSPACE} = \text{NPSpace}$.

Note 17. Space can be reused but time cannot be.